# Mutation Strategies for a Single-Objective Genetic Algorithm in Automated Timetable Generation

Your Name

April 26, 2026

**Abstract**

This report documents the mutation operators designed for a single-objective evolutionary algorithm (SOEA) applied to the university course timetabling problem. The algorithm searches for feasible and optimal weekly schedules by iteratively improving an initial population through selection, crossover, and mutation. We present four problem-specific mutation operators that exploit the hierarchical structure of a timetable while preserving or swiftly repairing hard constraints. The operators are Slot-Swap, Course-Relocate, Room-Change, and Block-Permute.

## 1 Introduction

Automated timetabling is a classic combinatorial optimisation problem characterised by a large search space and numerous interacting constraints. A genetic algorithm (GA) encodes candidate schedules as chromosomes, evolves a population, and uses mutation to maintain diversity and escape local optima. The choice of mutation operators profoundly affects the algorithm's ability to find high-quality feasible timetables.

This report focuses on the mutation module of a single-objective GA that minimises a weighted sum of three penalty components:

- **PTP** – Course time preference mismatch,
- **RTR** – Student room-to-room travel distance between consecutive slots,
- **CTRR** – Inconsistency in room and time slot assignment across the week.

All constraints (professor availability, room capacity, student group conflicts, etc.) are treated as hard constraints; any solution that violates them is rejected during initialisation and repaired or discarded after mutation.

## 2 Chromosome Representation

A timetable is represented as a nested dictionary:

```
time_table[day][time_slot] = list of Course instances (with assigned room)
```

Days are Monday to Friday. Each course object carries a reference to its student group, instructor, required number of slots, session type (lecture/lab/tutorial), and an associated room object.

# 3 Genetic Algorithm Outline

1. **Initialisation:** Generate a population of valid timetables using a smart greedy heuristic.
2. **Evaluation:** Assign a fitness value (total penalty) to each individual.
3. **Selection:** Choose parents via binary tournament based on fitness (minimisation).
4. **Crossover:** (to be defined)
5. **Mutation:** Apply one of the four operators described below with a certain probability.
6. **Replacement:** Form the next generation, typically using elitism and the newly generated offspring.

# 4 Mutation Operators

Mutation is applied to each offspring individually. The GA randomly selects one of the following operators, each having an equal probability. If the mutated timetable violates any hard constraint, the change is reverted (the original individual is retained). This simple rejection strategy avoids expensive repair procedures while still allowing neighbourhood exploration.

## 4.1 Slot-Swap

**Algorithm 1:** Slot-Swap Mutation

**Input:** A timetable $T$
**Output:** Mutated timetable $T'$

Pick two distinct (day, slot) pairs $(d_1, s_1)$ and $(d_2, s_2)$ where both slots are non-empty.;
Randomly choose one course from each slot: $c_1 \leftarrow T[d_1][s_1][i_1]$, $c_2 \leftarrow T[d_2][s_2][i_2]$.;
Swap the positions: $T[d_1][s_1][i_1] \leftarrow c_2$, $T[d_2][s_2][i_2] \leftarrow c_1$.;
(Each course retains its original room; the room is part of the course object.)

This operator perturbs the allocation of two courses while preserving the number of courses in each slot and their room assignments. It is particularly effective for escaping local optima where two courses are blocking a better global arrangement.

## 4.2 Course-Relocate

**Algorithm 2:** Course-Relocate Mutation

**Input:** A timetable $T$
**Output:** Mutated timetable $T'$

Find a randomly chosen occupied slot $(d, s)$ and a random index $i$ in $T[d][s]$.;
Let $c \leftarrow T[d][s][i]$ and remove $c$ from that slot.;
Choose a new day $d'$ and time slot $s'$ uniformly at random.;
Select a suitable room $r$ for $c$ from the available rooms (using the *GetAvailableRooms* function) that does not violate capacity or lab constraints.;
If no such room exists, revert the removal.;
Else set $c.room \leftarrow r$ and append $c$ to $T[d'][s']$.

Relocate moves a single course to a completely new slot and room, which can dramatically change the fitness landscape. The feasibility check (via room selection) ensures the new placement does not introduce an immediate hard constraint violation.

### 4.3 Room-Change

**Algorithm 3:** Room-Change Mutation

**Input:** A timetable $T$
**Output:** Mutated timetable $T'$

Pick a random occupied slot $(d, s)$ and a random course $c$ within it.;
Obtain the set of rooms $R$ that are suitable for $c$ (same constraints as above) *except* the currently assigned room.;
If $R \neq \emptyset$, set $c.room \leftarrow$ random choice from $R$.

Changing only the room of a course leaves the time allocation unchanged, thus it primarily affects the room-transition penalty (RTR) and the room-consistency penalty (CTRR). It is a fine-tuning operator that helps the GA converge to a precise room assignment.

### 4.4 Block-Permute

**Algorithm 4:** Block-Permute Mutation (for multi-slot courses)

**Input:** A timetable $T$
**Output:** Mutated timetable $T'$

Select a course $c$ that requires $n > 1$ slots.;
Identify the list of (day, slot) pairs where $c$ currently appears.;
**if** *$c$ requires continuous slots* **then**
  /* [ */
  Contiguous block shift] Let the current block be $[s_{start}, s_{start} + n - 1]$ on day $d$.;
  Randomly choose a new starting slot $s'$ on the same day such that all $n$ consecutive slots are free of conflicts (or we accept the move and let the constraint check revert it if needed).;
  Move $c$ from the old slots to the new slots.;
**else**
  /* [ */
  Non-continuous redistribution] Collect all current slots; randomly pick $n$ new (day, slot) pairs that are either empty or can accommodate $c$ without immediate conflict.;
  Relocate $c$ to those $n$ slots.;

Many courses (especially lectures or labs) require multiple slots per week, sometimes consecutive. Block-Permute respects the integrity of these units and explores different weekly patterns. The continuous variant preserves the contiguity requirement and simply shifts the block's starting time; the non-continuous variant redistributes individual occurrences.

## 5 Integration and Robustness

All mutation operators are designed to operate on a deep copy of the original timetable. After mutation, the hard-constraint checker is invoked. If any violation is detected, the mutation is discarded and the original individual is passed to the next generation unchanged. This conservative approach balances exploration with feasibility, ensuring the population never contains invalid timetables.

## 6 Conclusion

The four mutation operators exploit the structural properties of the timetable problem: temporal swapping, course relocation, room fine-tuning, and block-level rearrangement. Together they

provide a diverse set of local moves that prevent premature convergence and enable the GA to navigate the constrained search space effectively.

Future work will extend the algorithm to a multi-objective formulation (MOEA) and a co-operative co-evolutionary approach (CCEA), where the mutation operators will remain a crucial component of the evolutionary engine.